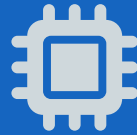


MODELAGEM COMPUTACIONAL



Alfabetos, Strings e Linguagens

Prof. Dr. José Jairo de S. e Silva

Aula 02

Alfabetos, Strings e Linguagens

OBJETIVOS DA AULA

🎯 Objetivos de Aprendizagem

Ao final desta aula, o estudante será capaz de:

Compreender os conceitos fundamentais de **alfabetos**, **strings** e **linguagens formais**, utilizando-os para formalizar matematicamente problemas computacionais.

Conteúdos:

- Alfabetos e símbolos
- Strings: definição e operações
- Substring, concatenação e reversa
- Tamanho de strings e string nula
- Potências de alfabetos e Σ^*
- Linguagens formais
- Fecho de Kleene

1. ALFABETOS E STRINGS

A Motivação

Algoritmos, computadores e problemas computacionais são objetos matemáticos complexos.

Nosso primeiro passo será definir os **conjuntos elementares de símbolos** que usaremos para construir tais objetos.

Estes elementos básicos serão chamados de **símbolos**, e conjuntos finitos destes símbolos serão chamados de **alfabetos**.

A construção acontece em camadas:

- **Símbolos** → elementos atômicos
- **Alfabetos** → conjuntos finitos de símbolos
- **Strings** → sequências de símbolos
- **Linguagens** → conjuntos de strings

A Motivação (cont.)

Analogia com construção:

Assim como um edifício é construído com tijolos, objetos matemáticos complexos são construídos a partir de símbolos.

Um único tijolo não faz um edifício: um único símbolo não descreve um objeto complexo.

Definição 2.1.1: Alfabeto e Símbolos

Definição 2.1.1: Um **alfabeto** é um conjunto finito qualquer, e **símbolos** são elementos deste conjunto.

Exemplo 2.1: Considere o conjunto $X = \{a, b, c, d\}$.

Os elementos a, b, c, d do conjunto X são chamados de **símbolos** do alfabeto X .

Note que o conceito de alfabeto nada mais é do que um sinônimo para o conceito matemático de **conjunto finito**.

Alfabetos comuns:

- $\Sigma = \{0, 1\}$: **alfabeto binário**
- $\{a, b, \dots, z\}$: letras minúsculas
- $\{0, 1, \dots, 9\}$: dígitos decimais

O Alfabeto Binário

O alfabeto mais usado neste curso:

$$\Sigma = \{0, 1\}$$

conhecido como **alfabeto binário**.

Usaremos Σ para denotar alfabetos; por convenção, quando não definido explicitamente, $\Sigma = \{0, 1\}$.

</> O Alfabeto ANSI C

Um exemplo de alfabeto bastante comum no mundo das linguagens de programação é:

$$\Sigma_{ANSIC} = \{0, 1, 2, \dots, 9, a, b, \dots, z, A, B, \dots, Z, \dots, \&, \#, \%, >, <, \dots\}$$

Este é o conjunto de todos os **caracteres válidos em um programa escrito em linguagem C** no padrão ANSI C.

Não vamos enumerá-lo símbolo a símbolo aqui, mas ele inclui:

- Letras maiúsculas e minúsculas
- Dígitos de 0 a 9
- Operadores e pontuação: **+**, **-**, *****, **/**, **{**, **}**, **;**, ...
- Caracteres especiais: espaço, tab, quebra de linha

</> O Alfabeto ANSI C: Por que importa?

Um **programa em C** pode ser visto matematicamente como uma **string** sobre o alfabeto Σ_{ANSIC} .

Essa formalização nos permite tratar programas como objetos matemáticos precisos, permitindo raciocinar rigorosamente sobre eles!

Definição 2.1.2: Strings

Definição 2.1.2: Dado um alfabeto Σ , uma **string** sobre Σ é uma **sequência de símbolos de Σ justapostos**.

Exemplos de strings:

- **00101** é uma string sobre $\Sigma = \{0, 1\}$
- ***abc*** é uma string sobre $\{a, b, c\}$
- Um programa em C é uma string sobre Σ_{ANSIC}

Um programa em linguagem C é uma **sequência de símbolos ANSI**, incluindo os símbolos especiais usados para quebra de linha, indentação e espaçamento.

Notação 2.1: Convenções

Em geral usamos a letra grega Σ para denotar alfabetos.

Quando o alfabeto não estiver explicitamente definido, Σ denotará, **por convenção**, o alfabeto binário $\{0, 1\}$.

Strings podem ter comprimento zero! A **string vazia** é denotada por ϵ .

⚠ Não confunda: símbolo, string e linguagem

1. **Símbolo**: é um elemento do alfabeto. Ex:

- $0 \in \Sigma$
- $1 \in \Sigma$.

2. **String**: é uma sequência finita de símbolos. Ex:

- $01 \in \Sigma^*$
- $1011 \in \Sigma^*$.

3. **Linguagem**: é um conjunto de strings. Ex:

- $L = \{01, 10, 111\}$, logo $01 \in L$ e $L \subseteq \Sigma^*$.

Atenção: ε (string vazia) $\neq \emptyset$ (conjunto vazio).

 Atenção: ε vs. $\emptyset$

A string vazia ε existe e tem tamanho 0.

Não confundir com $\emptyset$ (conjunto vazio, uma linguagem sem nenhuma string)!

- $\{\varepsilon\}$ é uma linguagem com **uma** string (a string vazia)
- $\emptyset$ é uma linguagem com **nenhuma** string

Definição 2.1.3: Substring e Concatenação

Definição 2.1.3: Uma **substring** de w é uma subsequência de símbolos de w . A **concatenação** de duas strings x e y , denotada xy , é a string resultante da justaposição das strings x e y .

Exemplo de substrings:

As strings ab , bb e bc são algumas substrings de $abbbbc$.

Exemplo de concatenação:

Considere $x = 111$ e $y = 000$. Então:

$$xy = 111000$$

Observe que a concatenação não é comutativa:

$$yx = 000111 \neq xy$$

Concatenação e Potência de Strings

Concatenação de uma string consigo mesma:

A partir de $y = 000$, temos:

$$yyy = 000000000$$

Notação: Quando escrevemos a^n , onde $a \in \Sigma$, queremos dizer n símbolos a consecutivos.

De maneira análoga, w^n denota a **concatenação de n cópias** da string w .

Exemplo: $1^5 = 11111$ e se $w = 10$, então $w^6 = 101010101010$.

Definição 2.1.4: Tamanho de uma String

Definição 2.1.4: O **tamanho** de uma string w é o número de símbolos de w , e é denotado por $|w|$. A **string nula** (sem nenhum símbolo) é denotada por ε .

Exemplo 2.2: Dadas as strings $aabc$, 001 e ε :

$$|aabc| = 4, \quad |001| = 3, \quad |\varepsilon| = 0$$

Propriedades importantes:

- $|\varepsilon| = 0$
- $|xy| = |x| + |y|$
- $|w^n| = n \cdot |w|$
- $x^0 = \varepsilon$ para qualquer string x

Definição 2.1.5: Reversa de uma String

Definição 2.1.5: A **reversa** de uma string w , denotada w^R , é a string lida de trás para frente. Por exemplo, se $w = 1110$, então $w^R = 0111$.

Formalizamos isso usando uma **definição indutiva (recursiva)**:

Base: Se $w = \varepsilon$, então $w^R = \varepsilon$.

Caso Geral: Seja w uma string com pelo menos um símbolo. Se $w = xa$, onde x é uma string e a é o **último símbolo** de w , então:

$$w^R = a \cdot x^R$$

Reversa: Verificação com Exemplo

Seja $w = abc$.

- $w = (ab)c$, logo $w^R = c \cdot (ab)^R$
- $(ab)^R = b \cdot (a)^R = ba$
- Portanto: $w^R = cba \checkmark$

Em teoria da computação, definições indutivas são fundamentais, não apenas em demonstrações, mas nas próprias definições dos objetos.

Definição 2.1.6: Potência de um Alfabeto

Definição 2.1.6: Dado um alfabeto Σ , o conjunto Σ^i de strings w sobre Σ tal que $|w| = i$ é dito uma **potência de Σ** . Definimos $\Sigma^0 = \{\varepsilon\}$.

Exemplo: Dado o alfabeto binário $\Sigma = \{0, 1\}$:

$$\Sigma^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}$$

Observações:

- $\Sigma^0 = \{\varepsilon\}$ (apenas a string vazia)
- $\Sigma^1 = \{0, 1\}$ (os próprios símbolos)
- $\Sigma^2 = \{00, 01, 10, 11\}$

Para todo i natural, Σ^i é um **conjunto finito**.

Exemplo manual: construindo Σ^0 , Σ^1 e Σ^2

Considere o alfabeto

$$\Sigma = \{a, b\}$$

Então:

- $\Sigma^0 = \{\varepsilon\}$
- $\Sigma^1 = \{a, b\}$
- $\Sigma^2 = \{aa, ab, ba, bb\}$

Observe que:

- Σ^0 contém todas as strings de comprimento **0**
- Σ^1 contém todas as strings de comprimento **1**
- Σ^2 contém todas as strings de comprimento **2**

 Exemplo manual (cont.): Σ^* vs. Linguagem

Agora considere a linguagem

$$L = \{w \in \Sigma^* : w \text{ termina em } a\}$$

Exemplos:

- $a \in L$
- $ba \in L$
- $aba \in L$
- $bb \notin L$

Assim, Σ^* contém **todas** as strings possíveis, enquanto uma linguagem escolhe apenas as strings que satisfazem uma propriedade.

☰ Cardinalidade de Σ^i

Se $|\Sigma| = k$, então:

$$|\Sigma^i| = k^i$$

Para o alfabeto binário: $|\{0, 1\}^i| = 2^i$

Portanto, $|\Sigma^3| = 2^3 = 8$ strings, confirmando o exemplo anterior.

Definição 2.1.7: O Conjunto Σ^*

Definição 2.1.7: Dado um alfabeto Σ , o **conjunto de todas as strings** sobre Σ é definido como:

$$\Sigma^* = \bigcup_{i \geq 0} \Sigma^i$$

Ou seja, $\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots$

Importante: Σ^* é um conjunto **infinito**, que contém strings de todos os comprimentos possíveis.

Para o alfabeto binário:

$$\{0, 1\}^* = \{\varepsilon, 0, 1, 00, 01, 10, 11, 000, \dots\}$$

Note que $\varepsilon \in \Sigma^*$ para qualquer alfabeto Σ , pois $\Sigma^0 = \{\varepsilon\}$.

☰ Σ^* : Por que importa?

Σ^* é o "universo" de todas as possíveis strings sobre um alfabeto.

Qualquer **linguagem** L sobre Σ será um subconjunto de Σ^* :

$$L \subseteq \Sigma^*$$

⚡ Ordenando o Infinito: Ordem Lexicográfica

Como Σ^* é infinito, precisamos de uma maneira padronizada para listar suas infinitas strings. Utilizamos a **ordem lexicográfica** (padrão em Teoria da Computação).

Regra:

1. Primeiro, ordenamos pelo **tamanho** (comprimento) da string.
2. Em seguida, ordenamos de forma **alfabética** (como em um dicionário) para strings de mesmo tamanho.

Exemplo para $\Sigma = \{0, 1\}$:

$\epsilon, 0, 1, 00, 01, 10, 11, 000, 001, 010, 011, 100, 101, 110, 111, \dots$

Nota: Isso evita o problema lógico de tentar listar começando apenas com zeros infinitamente (ex: $0, 00, 000, \dots$) e nunca alcançar as strings com "uns".

2. LINGUAGENS

Definição 2.2.1: Linguagem

Definição 2.2.1: Seja Σ um alfabeto. Uma **linguagem** é um conjunto $L \subseteq \Sigma^*$.

Em outras palavras, dado um alfabeto, uma linguagem é um **conjunto qualquer de strings** sobre este alfabeto.

Exemplos simples:

- $L = \{1010, 11111, 0000001\}$ é uma linguagem sobre $\{0, 1\}$
- $A = \{a, aa, aaa, aaaa, \dots\} = \{a^i \mid i \geq 1\}$ é uma linguagem sobre Σ_{ANSI}

Strings e linguagens são objetos diferentes:

- Strings são **seqüências**
- Linguagens são **conjuntos** de strings

Linguagens Finitas e Infinitas

Linguagens podem ser finitas ou infinitas:

- $\{1010, 11111\}$: linguagem **finita**
- $\{a, aa, aaa, \dots\}$: linguagem **infinita**

Linguagens são compostas de strings com número finito de símbolos, mas o conjunto em si pode ser infinito!

☰ Exemplo 2.5: Linguagens Binárias Importantes

Dado $\Sigma = \{0, 1\}$, são linguagens sobre Σ :

$$L_{01} = \{\varepsilon, 01, 0011, 000111, \dots\} = \text{"Strings da forma } 0^n 1^n, \text{ para } n \geq 0\text{"}$$

$$L_{PAL} = \{\varepsilon, 0, 1, 00, 11, 010, 101, 1001, 0110, \dots\} = \text{"Palíndromos"}$$

Um **palíndromo** é uma string que se lê da mesma forma da esquerda para a direita e da direita para a esquerda.

$$L_P = \{10, 11, 101, 111, 1011, \dots\} = \text{"Números primos em binário"}$$

$$L_{SQ} = \{0, 1, 100, 1001, 10000, \dots\} = \text{"Quadrados perfeitos em binário"}$$

Estas linguagens serão recorrentes ao longo do curso, pois cada uma apresenta um **nível diferente de complexidade computacional!**

🗨️ Linguagens Naturais e Formais

Analogia com linguagens do dia a dia:

Seja $\Sigma_P = \{a, b, c, \dots, z\}$ (letras do português).

Podemos pensar em L como o conjunto de todas as **palavras da língua portuguesa**:

- $bola \in L, caneta \in L, institucional \in L$

Com um símbolo separador $/$, poderíamos montar frases como $a / bola / verde / escura$.

Definir L matematicamente não é trivial! Poderíamos fixar um dicionário específico, mas definir o conjunto L' de todas as **frases válidas** seria bem mais difícil.

Nosso foco **não** serão linguagens naturais, mas sim linguagens **formais** como as de programação.

Linguagens Formais na Prática

Seja Σ_{ANSIC} o alfabeto ANSI C. Define-se:

$$L_C = \text{"conjunto de todos os programas válidos em C"}$$

Esta linguagem é **matematicamente precisa**: pode ser definida usando o conceito de **gramática formal**, que veremos adiante.

Compiladores e interpretadores de linguagens de programação são, em essência, **reconhedores de linguagens formais**!

⚙ Problemas Computacionais como Linguagens

O que significa "o problema de testar se um número n é primo"?

Formalização: Dado w , decidir se w é a representação binária de um número primo, ou seja, testar se $w \in L_P$.

A ideia central:

A linguagem L_P **incorpora a propriedade** "ser primo":

- Todo objeto em L_P tem essa propriedade
- Nenhum objeto fora de L_P tem essa propriedade

Responder se um número é primo **se reduz** a decidir se uma string pertence a L_P .

Em teoria da computação: **linguagens são sinônimos de problemas computacionais!**

Notação 2.2: Representação Binária

O número natural representado pela string binária w é denotado por $N(w)$.

Exemplos: $N(101) = 5$ e $N(1111) = 15$.

Com isso, podemos definir:

$$L_P = \{w \mid N(w) \text{ é primo}\}$$

$$L_3 = \{w \mid N(w) \text{ é múltiplo de } 3\}$$

💡 Refletindo: Por que Linguagens Representam Problemas?

A intuição fundamental:

No funcionamento interno de um computador, qualquer objeto matemático, em última análise, é uma **sequência de bits**.

Quando escrevemos um programa para testar se um número é primo, o que o programa faz é verificar se uma **sequência de bits na memória** representa ou não um número primo em binário.

Ou seja, o programa está testando se uma **string do alfabeto binário** pertence ou não à linguagem L_P .

💡 Generalizando: Problemas e Linguagens

Esse raciocínio vale para **qualquer problema** em que o objetivo é testar se um dado objeto matemático possui uma certa propriedade.

Problema	Linguagem equivalente
n é primo?	$w \in L_P?$
n é múltiplo de 3?	$w \in L_3?$
n é quadrado perfeito?	$w \in L_{SQ}?$
w é palíndromo?	$w \in L_{PAL}?$

Em boa parte do curso, o uso de **linguagens** será o formalismo padrão para representar problemas computacionais.

3. OPERAÇÕES COM LINGUAGENS

Definição 2.2.2: Concatenação de Linguagens

Definição 2.2.2: A **concatenação** de duas linguagens L e M é o conjunto de todas as strings formadas tomando-se $x \in L$ e $y \in M$ e fazendo a concatenação xy . Denotamos por LM ou $L \cdot M$.

Exemplo 2.8:

$L = \{00, 0\}$ e $L' = \{1, 111\}$:

$$LL' = \{001, 00111, 01, 0111\}$$

Exemplo 2.9:

$L = \{a, aa, aaa, \dots\}$ e $M = \{\varepsilon, x, xx, xxx, \dots\}$:

$$LM = \{a, ax, axx, \dots, aa, aax, aaxx, \dots\}$$

Exercício Resolvido 2.1

Seja $L_3 = \{w \mid N(w) \text{ é múltiplo de } 3\}$ e $R = \{0\}$.

Qual é L_3R ?

Adicionar **0** ao final de um número binário **dobra** o número original.

Portanto:

$$L_3R = \{w \mid N(w) \text{ é múltiplo de } 6\}$$

Exercício Resolvido 2.2

Exercício Resolvido 2.2: Seja $R = \{0\}$. Quais são as linguagens Σ^*R e Σ^*RR ?

Solução:

As strings binárias terminadas em **0** são precisamente os **múltiplos de 2** em binário.

Ao multiplicarmos por 2 novamente, obtemos múltiplos de 4. Portanto:

$$\Sigma^*R = \{w \mid N(w) \text{ é múltiplo de } 2\}$$

$$\Sigma^*RR = \{w \mid N(w) \text{ é múltiplo de } 4\}$$

Exercício Resolvido 2.2: Intuição

Concatenar $R = \{0\}$ ao final de uma linguagem equivale a **multiplicar por 2** cada número representado.

Concatenar R duas vezes equivale a **multiplicar por 4**.

Em geral, concatenar R exatamente k vezes equivale a **multiplicar por 2^k** .

Definição 2.2.3: Fecho de Kleene

Definição 2.2.3: O **fecho de Kleene** de L , denotado L^* , é o conjunto de strings formadas tomando qualquer número de strings de L (com repetições) e as concatenando:

$$L^* = \bigcup_{i \geq 0} L^i, \quad L^0 = \{\varepsilon\}, \quad L^i = \underbrace{L \cdot L \cdots L}_{i \text{ vezes}}$$

Exemplo 2.10:

Seja $L = \{000, 111\}$. Então:

$$L^* = \{\varepsilon, 000, 111, 000000, 000111, 111000, 111111, \dots\}$$

+ Definição 2.2.4: Fecho Positivo (L^+)

Enquanto o Fecho de Kleene (L^*) concatena a linguagem zero ou mais vezes, o **Fecho Positivo** (L^+) exige **pelo menos uma** concatenação.

$$L^+ = \bigcup_{i \geq 1} L^i = L \cdot L^*$$

Diferença Prática: O fecho positivo exclui a string vazia ϵ , **a menos** que a própria linguagem base L já contenha ϵ .

- Se $L = \{0\}$, $L^* = \{\epsilon, 0, 00, 000, \dots\}$
- Se $L = \{0\}$, $L^+ = \{0, 00, 000, \dots\}$

Fecho de Kleene: Exercício e Observação

Exercício Resolvido 2.3:

Considere $L = \{1\}$ e $R = \{0\}$. Qual é LR^* ?

$$LR^* = \{w \mid N(w) \text{ é potência de } 2\}$$

Importante: dois significados de A^* :

Se A é um **alfabeto**: A^* é a união de todas as potências do alfabeto (todas as strings possíveis).

Se A é uma **linguagem**: A^* é o fecho de Kleene (concatenações de strings de A).

Em **ambos** os casos, A^* é uma linguagem, ou seja, o conjunto de todas as strings construídas usando os elementos de A como "tijolos básicos".

✓ Resumo da Aula

- **Alfabeto:** conjunto finito de símbolos, o "tijolo básico" das strings.
- **String:** sequência de símbolos de um alfabeto; a string vazia é ε .
- **Operações com strings:** tamanho $|w|$, concatenação xy , reversa w^R , potência w^n .
- **Potência do alfabeto Σ^i :** conjunto de todas as strings de comprimento i .
- **Σ^* :** conjunto de **todas** as strings sobre Σ (infinito).
- **Linguagem:** qualquer subconjunto $L \subseteq \Sigma^*$.
- **Linguagens binárias importantes:** L_{01} , L_{PAL} , L_P , L_{SQ} .
- **Problemas computacionais** são formalizados como linguagens.
- **Concatenação de linguagens:** $LM = \{xy \mid x \in L, y \in M\}$.
- **Fecho de Kleene L^* :** todas as concatenações possíveis de strings de L .

EXERCÍCIOS

❓ Verificação rápida

Considere o alfabeto $\Sigma = \{0, 1\}$ e a linguagem

$$L = \{w \in \Sigma^* : \text{o número de símbolos 1 em } w \text{ é par}\}$$

Decida quais strings pertencem a L :

- | | |
|-----------------|--------|
| • ε | • 11 |
| • 0 | • 101 |
| • 1 | • 1010 |

Dica: conte quantas vezes o símbolo **1** aparece em cada string.

Resposta esperada:

Pertencem a L : $\varepsilon, 0, 11, 1010, 101$

Não pertencem a L : 1

Exercícios: Alfabetos e Strings (I)

Exercício 2.1 Seja $\Sigma = \{0, 1\}$, $x \in \Sigma^*$ e $y \in \{a, b, c\}^*$. Responda verdadeiro ou falso:

(a) Se w é uma string da forma $x01$, então $|w| > 2$?

(b) Se w é da forma $x010$, então $|w| \geq 3$?

(c) Se w é da forma $y010$, então $w \in \Sigma^*$?

Exercício 2.2 Seja Σ um alfabeto e i um número natural. Qual é a cardinalidade de Σ^i ?

Exercício 2.3 Forneça uma **definição formal indutiva** para x^n (concatenação de n cópias de x).

Exercício 2.4 É verdade que $x^0 = y^0$ para qualquer par de strings x e y ?

Exercício 2.5 Apresente uma **definição recursiva** para o conjunto Σ^* .

Exercício 2.6 Seja Σ um alfabeto, i um número natural e $w \in \Sigma^*$. Mostre que $(w^i)^R = (w^R)^i$.

Exercícios: Linguagens e Operações (I)

Exercício 2.7 Seja Σ um alfabeto arbitrário. Verdadeiro ou falso?

(a) Σ é uma linguagem?

(b) $\emptyset$ é uma linguagem?

(c) $\{\epsilon\}$ é uma linguagem?

(d) Σ^4 é uma linguagem?

Exercício 2.8 Forneça uma definição formal para a linguagem sobre $\Sigma = \{0, 1\}$ das strings que representam **múltiplos de 4** em binário.

Exercícios: Linguagens e Operações (II)

Exercício 2.9 A linguagem $\{w \mid \exists x \in \Sigma, w = xx^R\}$ é a linguagem de todos os palíndromos sobre Σ ? Em caso negativo, forneça um contra-exemplo e uma definição formal adequada.

Exercício 2.10 Seja $L_3 = \{w \mid N(w) \text{ é múltiplo de } 3\}$ e $R = \{\varepsilon, 0\}$. Qual é L_3R ?

Exercícios: Linguagens e Operações (III)

Exercício 2.11 Nas questões abaixo, A é sempre um alfabeto e L é sempre uma linguagem. Responda verdadeiro ou falso e justifique:

- Para todo A , é verdade que $A = A^*$?
- Existe A tal que $A = A^*$?
- Para todo A , é verdade que $A^* = (A^*)^*$?
- Para todo L , é verdade que $L^* = (L^*)^*$?
- Para todo $L \subseteq A^*$, é verdade que $A^* = L^*$?
- Existe uma linguagem L sobre A tal que $A^* = L^*$?

Exercício 2.9 (A linguagem dos palíndromos $w = xx^R$):

- **Falso.** Essa definição matemática gera apenas palíndromos de tamanho **par** (ex: se $x = ab$, $w = abba$). Ela não consegue gerar palíndromos de tamanho ímpar como aba ou 101 .
- *Definição correta:* $\{w = x\sigma x^R \mid x \in \Sigma^* \text{ e } \sigma \in \Sigma \cup \{\varepsilon\}\}$.

Exercício 2.10 (Concatenação L_3R com $R = \{\varepsilon, 0\}$):

- A operação gera as linguagens unidas: $L_3\{\varepsilon\} \cup L_3\{0\}$.
 - $L_3\{0\}$ converte múltiplos de 3 em múltiplos de 6.
 - $L_3\{\varepsilon\}$ mantém os números como múltiplos de 3 originais.
 - Como todo múltiplo de 6 **já é** um múltiplo de 3, a união resulta na linguagem original.
- Resposta final: O próprio L_3 .

PRÓXIMA AULA: UM GOSTINHO DO QUE VEM POR AÍ

De Linguagens a Máquinas

Nesta aula definiremos **alfabetos**, **strings** e **linguagens**. Vimos que:

- Problemas computacionais podem ser formalizados como **linguagens**
- Uma linguagem é um subconjunto de Σ^*
- A pergunta fundamental é: dada uma string w , ela pertence a L ?

Mas **quem** responde essa pergunta? Precisamos de um **modelo de computação**, ou seja, uma máquina que receba uma string como entrada e diga **SIM** ou **NÃO**.

Na próxima aula estudaremos os **Autômatos Finitos Determinísticos (DFAs)**: máquinas que usam exatamente os conceitos que aprendemos hoje (alfabetos, strings, linguagens) para processar entradas e tomar decisões.

Mas antes, vamos ver um exemplo concreto para ter uma intuição...

Uma Máquina de Vender Chocolates

Imagine uma máquina que vende chocolates que custam **R\$6** (6 reais). A máquina aceita apenas moedas de **R\$1** e **R\$2**.

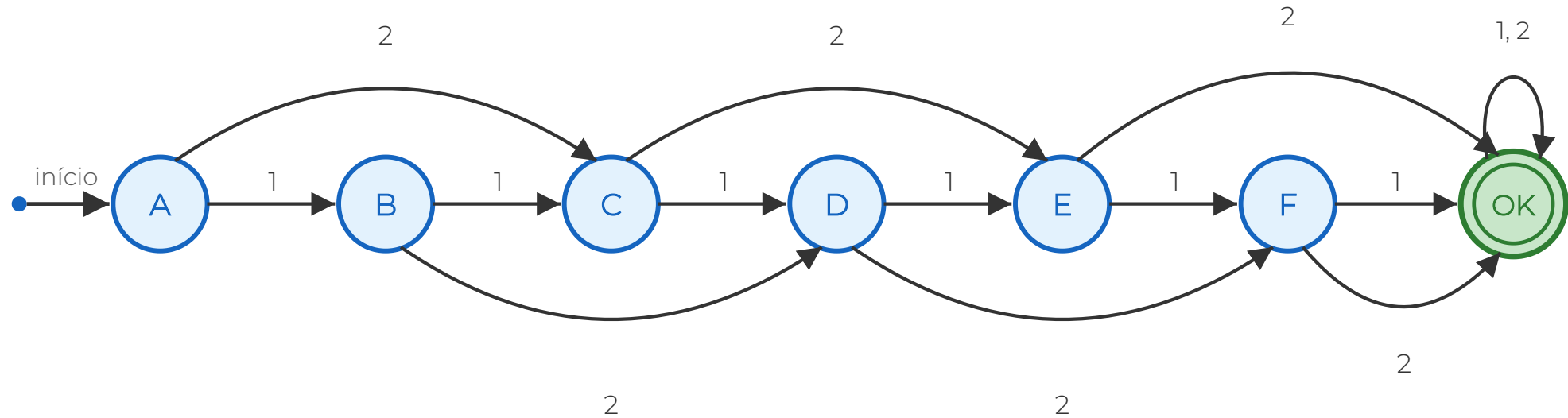
A máquina libera o chocolate **se, e somente se**, a sequência de moedas inseridas somar **pelo menos R\$6**.

Perguntas naturais:

- Se inserirmos R\$1 e depois R\$2, quanto já foi acumulado? (**R\$3**)
- A máquina libera o chocolate com a sequência R\$2, R\$2, R\$1? (**Não**, soma = R\$5)
- E com R\$2, R\$2, R\$2? (**Sim**, soma = R\$6)

Como podemos **modelar matematicamente** o funcionamento interno desta máquina?

O Autômato da Máquina de Chocolates



O Diagrama da Máquina de Chocolates

A ideia é representar a máquina como um **diagrama de estados**:

- Cada **estado** representa quanto dinheiro já foi acumulado
- Cada **transição** (seta) representa a inserção de uma moeda
- O estado **A** é o estado inicial (nenhuma moeda inserida, saldo = R\$0)
- O estado **OK** é o estado de aceitação (chocolate liberado!)

Estados e seus significados:

Estado	A	B	C	D	E	F	OK
Saldo	R\$0	R\$1	R\$2	R\$3	R\$4	R\$5	≥ R\$6

Exemplo: Se inserirmos R\$1 e depois R\$2, a máquina vai de **A** → **B** → **D** (saldo = R\$3).

O Diagrama da Máquina de Chocolates (cont.)

Algumas transições do diagrama:

- No estado **A**, ao inserir moeda de R\$1 → vai para **B**
- No estado **A**, ao inserir moeda de R\$2 → vai para **C**
- No estado **E**, ao inserir moeda de R\$2 → vai para **OK**
- No estado **OK**, qualquer moeda → permanece em **OK**

Observação: este é um cenário simplificado: a máquina **não devolve troco**! Se alguém inserir R\$2, R\$2, R\$2, R\$2 (total R\$8), o chocolate é liberado, mas os R\$2 extras são "engolidos".

O ponto central: este diagrama é o que chamamos de **máquina de estados** ou **autômato finito determinístico (DFA)**.

Na próxima aula, vamos formalizar este tipo de diagrama como um **objeto matemático preciso**, usando o conceito de **5-tupla**.

⚙️ O que é um Autômato Finito Determinístico (DFA)?

Um **Autômato Finito Determinístico** é definido formalmente como uma **5-tupla**:

$$D = (Q, \Sigma, \delta, q_0, F)$$

Mas o que significa cada componente desta 5-tupla?

Pense assim: para descrever completamente qualquer máquina de estados, precisamos especificar exatamente **5 ingredientes**. Nem mais, nem menos.

▮ Vamos ver cada um deles em detalhe.

⚙️ Os 5 Ingredientes da 5-Tupla

1. $Q \rightarrow$ Conjunto de estados

São todas as "situações" possíveis em que a máquina pode se encontrar.

Na máquina de chocolates: $Q = \{A, B, C, D, E, F, OK\}$

Cada estado representa uma configuração interna da máquina (quanto dinheiro foi acumulado).

2. $\Sigma \rightarrow$ Alfabeto de entrada

É o conjunto de símbolos que a máquina pode receber como entrada. Cada símbolo representa um "evento" que a máquina processa.

Na máquina de chocolates: $\Sigma = \{1, 2\}$

O símbolo **1** representa uma moeda de R\$1, e o símbolo **2** representa uma moeda de R\$2. A "entrada" da máquina é uma **string de moedas**, como **1, 2, 2, 1**.

⚙️ Os 5 Ingredientes da 5-Tupla (cont.)

3. $\delta \rightarrow$ Função de transição ($\delta : Q \times \Sigma \rightarrow Q$)

Esta é a "regra do jogo": dado o estado atual e o símbolo lido, δ diz para qual estado a máquina vai.

Na máquina de chocolates, alguns exemplos:

$$\delta(A, 1) = B \quad (\text{no estado A, ao receber R\$1, vai para B})$$

$$\delta(C, 2) = E \quad (\text{no estado C, ao receber R\$2, vai para E})$$

$$\delta(OK, 1) = OK \quad (\text{no estado OK, qualquer moeda mantém em OK})$$

▮ A função δ é definida para **cada par** (estado, símbolo). Ela é o coração do autômato!

⚙️ Os 5 Ingredientes da 5-Tupla (cont.)

4. $q_0 \in Q \rightarrow$ Estado inicial

É o estado em que a máquina começa antes de receber qualquer entrada.

Na máquina de chocolates: $q_0 = A$ (saldo = R\$0)

▮ Todo autômato tem **exatamente um** estado inicial.

5. $F \subseteq Q \rightarrow$ Conjunto de estados finais (ou de aceitação)

São os estados que indicam que a máquina "aceitou" a entrada. Se ao final do processamento a máquina estiver em um estado de F , a resposta é **SIM**.

Na máquina de chocolates: $F = \{OK\}$

▮ A máquina libera o chocolate se, e somente se, terminar no estado **OK**.

 Resumindo: A 5-Tupla Completa da Máquina de Chocolates

A máquina de chocolates é formalmente o DFA $C = (Q, \Sigma, \delta, A, F)$, onde:

Componente	Significado	Valor
Q	Estados	$\{A, B, C, D, E, F, OK\}$
Σ	Alfabeto	$\{1, 2\}$
δ	Transições	$\delta(A, 1) = B, \delta(A, 2) = C, \dots$
q_0	Estado inicial	A
F	Estados finais	$\{OK\}$

A beleza desta definição: com apenas 5 componentes, conseguimos descrever completamente e sem ambiguidade o funcionamento de qualquer máquina de estados!

Conectando com o que Aprendemos Hoje

Note a conexão com os conceitos desta aula:

- A **entrada** da máquina é uma **string** sobre o alfabeto $\Sigma = \{1, 2\}$
- O conjunto de **todas as entradas possíveis** é $\Sigma^* = \{1, 2\}^*$
- A **linguagem aceita** pela máquina é:

$$L(C) = \{w \in \{1, 2\}^* \mid \text{a soma dos símbolos de } w \geq 6\}$$

- $w = 2211 \in L(C)$, pois $2 + 2 + 1 + 1 = 6 \geq 6$
- $w = 221 \notin L(C)$, pois $2 + 2 + 1 = 5 < 6$
- $w = 222 \in L(C)$, pois $2 + 2 + 2 = 6 \geq 6$

 Linguagens, strings, alfabetos... tudo se conecta!

▶▶ Na Próxima Aula...

Estudaremos formalmente os **Autômatos Finitos Determinísticos (DFA)**:

- Definição completa da 5-tupla e como construir DFAs
- A **função de transição estendida** $\hat{\delta}$: como processar strings inteiras
- **Aceitação e rejeição** formal de strings
- O conceito de **linguagem regular**
- Construção de DFAs para diversas linguagens

Os DFAs são o primeiro modelo formal de computação que estudaremos, e a base para tudo que virá depois!